# Lock-Free Ring Buffer - SPSC Atomic Queue

## Theory, Edge Cases & Code Examples

- ✓ Ring Buffer Basics
- ✓ Single Producer, Single Consumer (SPSC)
- ✓ Memory Ordering
- ✓ Full vs Empty Disambiguation
- ✓ False Sharing

*→ Swipe to learn →*

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Theory

## 📌 1. Ring Buffer Basics

**Fixed-size circular queue:**

```
[_][_][_][_][_][_][_][_]
 ^head          ^tail
```

**Operations:**

- `push(item)`: Write at tail, advance tail
- `pop()`: Read at head, advance head

**Wrap-around:** Indices mod SIZE

---

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Theory

### 📌 2. Single Producer, Single Consumer (SPSC)

**Simplified:** Only one writer, one reader (most common).

**Key insight:** No contention between push/pop!

**Lock-free implementation:**
- `head_` (atomic, modified by consumer)
- `tail_` (atomic, modified by producer)

---

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Theory

## 📌 3. Memory Ordering (1/2)

**Producer (push):**

```
buffer_[tail] = item;  // Write data
tail_.store(new_tail, std::memory_order_release);  // Publish
```

**Consumer (pop):**

```
size_t head = head_.load(std::memory_order_acquire);  // See latest
item = buffer_[head];
```

**Release-acquire** ensures visibility.

---

```
#include <atomic>
#include <array>
#include <optional>

template<typename T, size_t SIZE>
class SPSCRingBuffer {
private:
    std::array<T, SIZE> buffer_;
    std::atomic<size_t> head_{0};
    std::atomic<size_t> tail_{0};

    size_t next_index(size_t current) const {
        return (current + 1) % SIZE;
    }

public:
    bool try_push(const T& item) {
        size_t current_tail = tail_.load(std::memory_order_relaxed);
        size_t next_tail = next_index(current_tail);

        if (next_tail == head_.load(std::memory_order_acquire)) {
            return false;  // Full
        }

        buffer_[current_tail] = item;
        tail_.store(next_tail, std::memory_order_release);
// ...continues on next slide
```

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Theory

## 📌 3. Memory Ordering (2/2)

[Code continues across slides]

```
// ...continued from previous slide

        return true;
    }

    std::optional<T> try_pop() {
        size_t current_head = head_.load(std::memory_order_relaxed);

        if (current_head == tail_.load(std::memory_order_acquire)) {
            return std::nullopt;  // Empty
        }

        T item = buffer_[current_head];
        head_.store(next_index(current_head), std::memory_order_release);

        return item;
    }

    bool empty() const {
        return head_.load(std::memory_order_acquire) ==
               tail_.load(std::memory_order_acquire);
    }

    bool full() const {
        return next_index(tail_.load(std::memory_order_acquire)) ==
               head_.load(std::memory_order_acquire);
    }

    size_t size() const {
        size_t h = head_.load(std::memory_order_acquire);
        size_t t = tail_.load(std::memory_order_acquire);

        if (t >= h) {
            return t - h;
        } else {
            return SIZE - h + t;
        }
    }
};
```

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Edge Cases

### 📌 1. Full vs Empty Disambiguation

**Problem:** `head == tail` could mean empty OR full.

**Solution:** Sacrifice one slot:

```
bool full() const {
    return next_index(tail) == head;  // One slot always empty
}
```

**Alternative:** Use separate size counter.

---

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Edge Cases

### 📌 2. False Sharing

**Problem:** head_ and tail_ on same cache line → contention.

**Solution:** Align to different cache lines:

```
alignas(64) std::atomic<size_t> head_{0};
alignas(64) std::atomic<size_t> tail_{0};
```

---

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Code Examples

### 📌 1. Producer-Consumer (1/2)

**This example demonstrates a classic producer-consumer pattern using the lock-free ring buffer for high-performance inter-thread communication.**

**What this code does:**
- Creates a shared SPSC ring buffer with 1024 slots between two threads
- Producer thread writes 10,000 integers sequentially into the buffer, spinning when full
- Consumer thread reads from the buffer, spinning when empty, and tracks progress
- Both threads run concurrently without any mutex locks or condition variables

**Key concepts demonstrated:**
- Lock-free communication eliminates context switching overhead from mutex contention
- Spin-waiting (busy-wait) is acceptable here because the buffer fills/empties quickly
- The ring buffer acts as a bounded queue with automatic wraparound
- Each thread only modifies its own index (producer: tail, consumer: head), preventing write conflicts

**Real-world applications:**
- Audio/video processing pipelines where latency must be minimal
- High-frequency trading systems where microseconds matter
- Real-time sensor data collection with processing on separate thread
- Game engines with render thread consuming data from game logic thread

**Performance implications:**
- Zero allocation after initialization (fixed-size buffer)
- No syscalls or context switches during normal operation
- CPU cores stay active (spinning), trading power efficiency for latency
- Cache-friendly sequential access pattern within each thread

---

```
#include <thread>
#include <iostream>

SPSCRingBuffer<int, 1024> buffer;

void producer() {
    for (int i = 0; i < 10000; ++i) {
// ...continues on next slide
```

# Lock-Free Ring Buffer - SPSC Atomic Queue

## Code Examples

### 📌 1. Producer-Consumer (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        while (!buffer.try_push(i)) {
            // Spin until space available
        }
    }
}

void consumer() {
    int count = 0;
    while (count < 10000) {
        if (auto item = buffer.try_pop()) {
            ++count;
            if (count % 1000 == 0) {
                std::cout << "Consumed " << count << '\n';
            }
        }
    }
}

int main() {
    std::thread prod(producer);
    std::thread cons(consumer);

    prod.join();
    cons.join();

    return 0;
}
```

# What You Learned

## Key Takeaways

- ✓ Ring Buffer Basics
- ✓ Single Producer, Single Consumer (SPSC)
- ✓ Memory Ordering
- ✓ Full vs Empty Disambiguation

→ *Swipe to learn* →